

Supabase Realtime Chat App

Complete Technical Documentation & Developer Guide

Next.js 14 · Supabase · TypeScript · Tailwind CSS

1. Project Overview

The Supabase Realtime Chat App is a production-grade, full-stack group chat application built with Next.js 14 and Supabase. It demonstrates how modern web applications can deliver instant, real-time experiences without polling or complex server infrastructure.

Think of it as a simplified Slack or Discord — multiple users can sign up, join named rooms (General, Random, Tech), and exchange messages that appear on every connected screen the moment they are sent, with zero page refreshes.

Core capabilities

- Multi-room group chat with instant message delivery
- Live presence system — see exactly who is online right now
- Per-user authentication with JWT session management
- Row Level Security — each user's data is isolated at the database level
- Message deletion with hover-to-reveal controls
- Grouped messages — consecutive messages from the same user are visually grouped
- Auto-scroll to latest message on new arrivals
- Reusable component architecture — Button, Input, Avatar, Badge used across the whole app

2. Technology Stack

Technology	Version	Purpose
Next.js	14 (App Router)	React framework, file-based routing, SSR/CSR hybrid
Supabase	Latest	Postgres DB, Auth, Realtime, Presence — all-in-one backend
TypeScript	5.x	Type safety across all components and API calls
Tailwind CSS	4.x	Utility-first styling with CSS custom properties
date-fns	4.x	Lightweight date formatting for message timestamps
DM Sans / DM Mono	Google Fonts	Custom typography — sans for UI, mono for code

3. Project File Structure

The project follows a strict separation of concerns using Next.js App Router conventions combined with a feature-based component architecture.

File / Folder	Type	Responsibility
app/page.tsx	Page	Login and signup entry point. Redirects to /chat if already authenticated.
app/layout.tsx	Layout	Root HTML shell, loads global CSS and Google Fonts.
app/globals.css	Styles	Tailwind import, CSS custom properties (design tokens), keyframe animations.
app/chat/page.tsx	Page	Room selection hub. Lists the three available chat rooms.
app/chat/[room]/page.tsx	Dynamic Page	The live chat room. Assembles Sidebar + MessageList + MessageInput + OnlineUsers.
lib/supabase.ts	Utility	Creates and exports the single Supabase client instance used everywhere.
lib/types.ts	Types	Shared TypeScript types: Message, PresenceUser, Room. Also exports ROOMS array.
components/ui/Button.tsx	UI Primitive	Reusable button with primary / ghost / danger variants and loading state.
components/ui/Input.tsx	UI Primitive	Reusable input with label, error, and forwardRef support.
components/ui/Avatar.tsx	UI Primitive	Initials-based avatar with consistent color derived from username hash.
components/ui/Badge.tsx	UI Primitive	Online/offline status dot and numeric count badge.
components/layout/Navbar.tsx	Layout	Top navigation bar with brand, current room name, user avatar, sign-out.
components/layout/Sidebar.tsx	Layout	Left panel listing all rooms. Highlights the active room.
components/auth/AuthForm.tsx	Feature	Complete sign-in / sign-up form using the reusable Input and Button components.
components/chat/MessageList.tsx	Feature	Loads initial messages, subscribes to Realtime, auto-scrolls.

File / Folder	Type	Responsibility
components/chat/MessageBubble.tsx	Feature	Renders a single message with avatar grouping and hover-to-delete.
components/chat/MessageInput.tsx	Feature	Typing box and send button. Inserts into Supabase on submit.
components/chat/OnlineUsers.tsx	Feature	Supabase Presence panel showing live online users in the current room.

4. Database Design

4.1 Messages Table

Every message sent in the app is stored as a single row in the messages table. The schema is intentionally minimal — Supabase generates the REST and Realtime APIs directly from it.

```
CREATE TABLE messages (
  id          UUID          DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id     UUID          REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  room        TEXT          NOT NULL DEFAULT 'general',
  content     TEXT          NOT NULL,
  username    TEXT          NOT NULL,
  created_at  TIMESTAMPTZ   DEFAULT now()
);
```

Column	Type	Purpose
id	UUID (PK)	Unique identifier auto-generated by Postgres gen_random_uuid()
user_id	UUID (FK)	Links to auth.users — enforces that every message belongs to a real user
room	TEXT	Room slug: 'general', 'random', or 'tech'
content	TEXT	The actual message body, max 1000 characters enforced in the UI
username	TEXT	Stored at insert time from user metadata — avoids a JOIN on every query
created_at	TIMESTAMPTZ	UTC timestamp. Used for sorting and display formatting with date-fns

4.2 Authentication — auth.users (Supabase managed)

Supabase manages the auth.users table internally. You never write to it directly. It stores the hashed password, email, user ID (UUID), email confirmation status, and user_metadata — a JSON field where we store the username chosen at signup.

5. Supabase Concepts Used

5.1 Authentication

Supabase Auth provides a complete identity system. This app uses Email + Password authentication with optional username metadata stored at signup time.

How it works in this app

- Sign Up: supabase.auth.signUp() creates the user, sends a confirmation email, and stores { username } in user_metadata
- Sign In: supabase.auth.signInWithPassword() validates credentials and returns a JWT access token
- Session: The token is automatically stored in localStorage and attached to every subsequent Supabase request
- Session check: supabase.auth.getSession() is called on page load to redirect authenticated users away from the login page
- Sign Out: supabase.auth.signOut() clears the token and redirects to login

```
// Sign up with username in metadata
await supabase.auth.signUp({
  email, password,
  options: { data: { username: 'Laiba' } }
})
```

5.2 Row Level Security (RLS)

Row Level Security is Postgres's built-in authorization system. When enabled, every query is automatically filtered by a policy — even if someone steals the anon key, they cannot read or modify data they are not supposed to.

Policies on the messages table

Policy Name	Operation	Rule	Effect
Logged in users can read	SELECT	auth.uid() IS NOT NULL	Any authenticated user can read all messages in any room
Users insert own messages	INSERT	auth.uid() = user_id	You can only insert rows where user_id matches your own auth ID
Users delete own messages	DELETE	auth.uid() = user_id	You can only delete your own messages — not others'

This means even if a malicious user calls the API directly with a custom payload, they cannot insert a message pretending to be someone else, and cannot delete another user's message.

5.3 Realtime — Postgres Changes

Supabase Realtime listens to the Postgres Write-Ahead Log (WAL). When a row is inserted, updated, or deleted, Supabase broadcasts that event to all WebSocket subscribers matching the filter.

How MessageList uses Realtime

- On mount, the component fetches the last 50 messages via a normal REST query
- It then opens a Realtime channel filtered to the current room
- When a new INSERT arrives, the new message is appended to state — React re-renders instantly
- When a DELETE arrives, the message is removed from state — it disappears from all screens
- On unmount (leaving the room), the channel is removed to prevent memory leaks

```
const channel = supabase
  .channel(`room-${room}`)
  .on('postgres_changes', {
    event: 'INSERT',
    schema: 'public',
    table: 'messages',
    filter: `room=eq.${room}`
  }, (payload) => {
    setMessages(prev => [...prev, payload.new as Message])
  })
  .subscribe()
```

5.4 Realtime — Presence

Presence is a Supabase Realtime feature that synchronizes shared state between clients — specifically who is currently connected to a channel. It uses WebSocket heartbeats: as long as a client has the tab open, it keeps sending 'I'm here' signals. When a tab closes or the network drops, the user disappears from the presence list within a few seconds.

How OnlineUsers uses Presence

- When a user enters a room, `channel.track()` broadcasts their `user_id` and `username`
- Every connected client receives a presence sync event
- The `presenceState()` map is converted to a list of unique online users
- The right sidebar re-renders showing exactly who is in the room right now
- When a user leaves, their entry is automatically removed from the state

```
// Track yourself as online
await channel.track({
  user_id: currentUser.id,
  username: currentUser.username,
  online_at: new Date().toISOString()
})
```

```
// Listen for presence changes
channel.on('presence', { event: 'sync' }, () => {
  const state = channel.presenceState()
  const users = Object.values(state).flat()
  setOnlineUsers(users)
})
```

5.5 Auto-generated REST API

Every Supabase project automatically generates a RESTful API from the database schema using PostgreSQL. The Supabase JS client wraps this API with a clean query-builder interface.

Operation	Code	What it does
SELECT	<code>supabase.from('messages').select('*').eq('room', room).order('created_at')</code>	Fetch last 50 messages in a room, sorted oldest first
INSERT	<code>supabase.from('messages').insert({ content, room, user_id, username })</code>	Send a new message — triggers Realtime for all subscribers
DELETE	<code>supabase.from('messages').delete().eq('id', id)</code>	Delete a specific message — triggers Realtime DELETE event

6. Complete Application Workflows

6.1 Sign Up Workflow

- Step 1 — User fills in username, email, and password in AuthForm.tsx and submits.
- Step 2 — `supabase.auth.signUp()` is called with email, password, and `{ username }` in `options.data`.
- Step 3 — Supabase creates a new row in `auth.users`, stores the hashed password, and saves username to `user_metadata`.
- Step 4 — A confirmation email is sent to the user's inbox (configurable in Supabase dashboard).
- Step 5 — The UI shows a success message: 'Check your email for a confirmation link'.
- Step 6 — After confirming, the user can sign in and a JWT session is established.

6.2 Sign In Workflow

- Step 1 — User enters email and password, submits the form.
- Step 2 — `supabase.auth.signInWithPassword()` sends credentials to Supabase Auth.
- Step 3 — Supabase validates, returns a JWT access token and refresh token.
- Step 4 — The client library stores the token in `localStorage` automatically.
- Step 5 — `useRouter().push('/chat')` redirects the user to the room selection page.
- Step 6 — All future Supabase calls automatically include the JWT — the user is 'logged in'.

6.3 Send a Message Workflow

- Step 1 — User types in the `MessageInput` box and presses Enter or clicks Send.
- Step 2 — `supabase.from('messages').insert()` is called with content, room, `user_id`, and username.
- Step 3 — Postgres inserts the row into the `messages` table and writes to the Write-Ahead Log.
- Step 4 — Supabase Realtime detects the INSERT event on the WAL.
- Step 5 — The event is broadcast through Websocket to all clients subscribed to that room's channel.
- Step 6 — Each subscriber's `onInsert` callback fires, appending the new message to their local state.
- Step 7 — React re-renders `MessageList` on all screens simultaneously — the message appears everywhere instantly.
- Step 8 — The input box is cleared and focus is returned to it, ready for the next message.

6.4 Presence / Online Users Workflow

Step 1 — When a user enters a room, `OnlineUsers.tsx` creates a Presence channel keyed by their `user_id`.

Step 2 — `channel.subscribe()` connects to the WebSocket. On success, `channel.track()` broadcasts the user's info to all peers.

Step 3 — Every other connected client receives a 'presence sync' event and updates their online users list.

Step 4 — The right sidebar re-renders showing the updated list of who is in the room.

Step 5 — Supabase sends periodic heartbeats. If a client disconnects, the heartbeat stops.

Step 6 — After a timeout, Supabase removes the disconnected user from the presence state and broadcasts another sync event.

Step 7 — All remaining clients update their sidebar — the user disappears from the online list.

7. Reusable Component Architecture

The app is built with a four-layer component hierarchy. Each layer has a clear responsibility and never reaches beyond its scope.

Layer	Folder	Components	Reusability
UI Primitives	<code>components/ui/</code>	Button, Input, Avatar, Badge	Used in every part of the app. No business logic. Props-only.
Layout	<code>components/layout/</code>	Navbar, Sidebar	Shared chrome that wraps pages. Receives data via props.
Feature	<code>components/chat/ + auth/</code>	AuthForm, MessageList, MessageBubble, MessageInput, OnlineUsers	Contain Supabase calls and business logic. Used in specific pages.
Pages	<code>app/</code>	<code>page.tsx</code> files	Compose features and layouts. Handle routing and auth guards.

Why this matters

- `Button.tsx` is written once and used in `AuthForm`, `MessageInput`, and `Navbar` — consistent styling everywhere
- `Avatar.tsx` generates a color from a username hash — the same user always gets the same color across the app

- Input.tsx handles label, placeholder, error display, and ref forwarding in one place — no duplication
- If you want to change the button style across the whole app, you change one file

8. Supabase Concepts Mastered

Concept	Where used	What you learned
supabase.auth.signUp()	AuthForm.tsx	Create users, pass metadata, handle email confirmation
supabase.auth.signInWithPassword()	AuthForm.tsx	Validate credentials, establish JWT session
supabase.auth.getSession()	All page.tsx files	Check auth state on load, implement route guards
supabase.auth.signOut()	Navbar.tsx	Clear session, redirect to login
supabase.from().select()	MessageList.tsx	Query rows with filters, ordering, and limits
supabase.from().insert()	MessageInput.tsx	Insert a row, trigger Realtime for all subscribers
supabase.from().delete()	MessageBubble.tsx	Delete own rows, trigger Realtime DELETE event
Row Level Security	Supabase dashboard	Isolate data per user at the database level with policies
Realtime postgres_changes	MessageList.tsx	Subscribe to INSERT/DELETE events on a table with filters
Realtime Presence	OnlineUsers.tsx	Track who is online, sync state across all connected clients
supabase.channel()	MessageList + OnlineUsers	Create named WebSocket channels, clean up on unmount
user_metadata	AuthForm + all pages	Store and retrieve extra user data (username) alongside auth

9. Setup & Running the Project

9.1 Supabase Setup

1. Create a free account at supabase.com and create a new project.
2. Go to SQL Editor and run the database setup script (creates messages table + RLS policies + enables Realtime).
3. Go to Settings → Data API and copy the Project URL and anon public key.

9.2 Local Setup

```
npx create-next-app@latest supabase-chat --typescript --tailwind --app
cd supabase-chat
npm install @supabase/supabase-js date-fns
```

Create `.env.local` in the project root:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project-id.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_anon_key_here
```

Copy all project files into the correct folders, then run:

```
npm run dev
```

Open `localhost:3000` in two different browser tabs. Sign up with two different accounts and join the same room — messages will appear on both screens instantly.

10. Key Takeaways

- Supabase eliminates the need to write a backend server for most CRUD + auth + realtime use cases
- Row Level Security moves authorization into the database — it cannot be bypassed from the frontend
- Supabase Realtime uses Postgres WAL, not a separate pub/sub system — your data and your events are in sync by design
- Presence is independent of the database — it runs on WebSocket state, not Postgres rows
- The Supabase JS client auto-attaches your JWT to every request — you never manually set auth headers
- Cleaning up channels on component unmount is essential — forgetting this causes memory leaks and duplicate events

